PersonaLint: Personalized Code Smell Detection Using Individual Developer Commit History as an Adaptive Baseline

Rajita S K
Dept. of Computer Science and Engineering
[Your College/University Name]
[City, Country]
rajita.shoba@gmail.com

Abstract

Static code smell detectors typically apply a single, universal threshold to all developers, regardless of individual coding style. This paper presents PersonaLint, a code smell detection approach that personalizes detection thresholds to each developer's own commit history rather than a population-wide norm. We mine 131,285 functions from 16 developers across 10 open-source Python repositories, extract five AST-derived structural metrics per function, and compare three detection strategies — a universal z-score threshold, a per-developer personalized z-score threshold, and a Random Forest classifier — against 196 functions hand-labeled by a human reviewer. Against this human-judgment ground truth, the personalized threshold achieved higher precision (0.570 vs. 0.513) and a lower false positive rate (31.2% vs. 39.9%) than the universal threshold, with comparable recall (0.983 vs. 1.000); a McNemar's test confirmed this improvement is statistically significant (p = 0.0127). A feature ablation further revealed that nesting depth, not function length, is the strongest predictor of human-perceived smell. We implement the personalized detector as a live VS Code extension, demonstrating that the same code is flagged differently depending on whose history it is evaluated against.

Keywords—*code smell detection, personalized software metrics, static analysis, developer coding style, abstract syntax tree, machine learning, software quality*

I. INTRODUCTION

Static code smell detectors — tools that flag functions as overly long, deeply nested, or otherwise poorly structured — almost universally apply a single, fixed threshold across an entire codebase or organization [26]. A function with cyclomatic complexity above 10, or more than 50 lines, is flagged for every developer identically, regardless of that developer's typical coding style. This one-size-fits-all approach can produce excessive false positives for developers whose natural style differs from the population norm, contributing to linter fatigue and reduced trust in automated code review tools.

This paper investigates whether personalizing the detection threshold to each developer's own commit history — rather than a population-wide average — improves detection quality. We build PersonaLint, a pipeline that mines a developer's git history, extracts structural features from every function they have written via Abstract Syntax Tree (AST) analysis, and constructs a personal "fingerprint": the mean and standard deviation of five structural metrics across that developer's own code. New or modified functions are then flagged only if they deviate significantly from that personal baseline, rather than from a fixed, universal cutoff.

Our central research question is: does conditioning the detection threshold on an individual developer's history improve precision and reduce false positives, relative to a universal threshold, without materially harming recall? We answer this empirically using 196 functions hand-labeled by a human reviewer, drawn from a corpus of 131,285 functions mined from 10 real-world open-source Python repositories.

The underlying motivation is economic as much as technical. Code review and static analysis triage consume a non-trivial share of a development team's time, and every warning a reviewer must manually dismiss as not-actually-a-problem is time spent not reviewing logic, correctness, or design. If a sizable fraction of flagged functions are only flagged because the detector's notion of "normal" does not match a specific developer's normal, personalization is not merely a usability nicety but a direct reduction in wasted reviewer effort, compounding across every commit that developer makes for the lifetime of the tool's deployment.

The contributions of this paper are: (1) an empirical comparison of universal and personalized detection thresholds against real human judgment, with a statistically significant result favoring personalization; (2) a feature ablation study revealing that nesting depth, not function length, best predicts human-perceived code smell; and (3) a working VS Code extension that demonstrates the personalized detector live, flagging the same code differently depending on whose history it is evaluated against.

II. RELATED WORK

Fowler's catalog of code smells [26] remains the conceptual foundation for most automated detection work, and the field has since grown into a substantial literature spanning datasets, classical machine learning, deep learning, large language models, and empirical studies of detection practice. We organize this related work by technique and purpose, and position PersonaLint's contribution — personalizing the detection threshold itself, rather than the model used to apply it — against each.

A. Datasets and Benchmarks

Progress in this area is shaped heavily by the datasets available for training and evaluation. Alhadeaf and Alrashoud [1] recently introduced a unified multi-label code smell dataset addressing the fragmentation of earlier single-label, single-smell resources. Madeyski and Lewowski's MLCQ [28] similarly contributed a dataset of professionally reviewed smell severity ratings, and observed substantial disagreement among professional reviewers on smell severity — early evidence that a single universal standard may not reflect how individual developers actually perceive code quality, a theme this paper tests directly. Palomba et al. [27] conducted a large-scale empirical study of code smell co-

occurrence and lifecycle, again relying on fixed, metric-based thresholds applied consistently across all developers in their corpus.

B. Machine Learning and Deep Learning Approaches

A large body of recent work applies supervised ML and deep learning to smell detection. Ibrahim and Ahmed's Smell-ML [4] targets under-explored design and structural flaws in enterprise systems; Sharma and Singh [5] propose interpretable deep learning for smell prioritization; Shin and Bae [9] evaluate deep neural models for structural vulnerability and smell co-occurrence; Li et al. [11] combine pre-training and stacking models for hybrid feature extraction; Mahmoud et al. [16] use a CNN and a neutrosophic-set technique for smell detection and ranking; and Romano et al.'s DeleSmell [19] is a deep learning ensemble model evaluated at ICSE. Zhang et al. [6] synthesize a decade of this work in a systematic review of supervised ML pipelines for code smell detection, and observe that the overwhelming majority of surveyed pipelines are trained and evaluated against a single, population-wide label distribution rather than per-developer or per-individual distributions — the specific gap this paper targets.

C. Large Language Models for Smell Detection

Very recent work applies large language models directly to this task. Zhang et al.'s PromptSmell [2] uses prompt learning with LLMs for both detection and refactoring validation; Gao et al.'s SmellDetector [3] frames multi-label agile design smell discovery as an LLM-based classification problem; and Ali and Baysal [15] propose a transformer-based framework built on contextual code representations. These approaches improve raw detection accuracy and reduce manual feature engineering relative to the AST-metric pipeline used in this paper, but, like the ML/DL work above, they still learn a single decision boundary shared across every developer represented in the training corpus.

D. Ensembles, Optimization, and Dynamic Detection

Sandouka and Aljamaan [10] evaluate heterogeneous ensembles for smell classification in dynamic codebases; Wang and Zhang [13] use a weighted cockroach swarm optimization algorithm to automatically generate structural threshold rules rather than hand-tuning them, an automated alternative to the fixed $k = 2.0$ threshold used as our primary comparison point; Dewangan et al. [24] improve ML-based detection through hyper-parameter optimization; and Prasad and Kumar's DPy [7] is a dynamic evaluation and detection tool specifically for Python implementation-level smells, the closest prior system to PersonaLint in both implementation language and scope.

E. Architectural and Domain-Specific Smells

Beyond function- and class-level smells, Cerny et al. [21] study context-driven detection of architectural smells in cloud-native applications, and Liu et al. [14] propose cross-project transfer learning for microservice-level smell discovery, both arguing that a single fixed rule set transfers poorly across deployment contexts — a structural analogue, at the architecture level, to the developer-level personalization problem addressed here.

F. Feature Engineering, Interpretability, and Prioritization

Pecorelli et al. [22] examine static analysis warnings as input features for ML-based smell detectors; Cruz et al. [23] evaluate structural metric weightings for detection algorithms, directly relevant to the feature ablation in Section IV; Santos and Ferreira [25] focus specifically on interpretable models for smell detection, motivating our inclusion of threshold-based detectors alongside the less interpretable Random Forest; and Mkaouer and Kessentini [18] address multi-objective refactoring recommendation based on severe design smell prioritization, a downstream step that PersonaLint's per-function flags are intended to feed.

G. Empirical and Survey Studies

Several works step back from individual techniques to characterize the field as a whole. Smith et al. [8] conduct a bibliometric study of code smell and technical debt evaluation metrics; Al-Kharabsheh and Fazal-e-Amin [12] present a systematic literature review of detection methods, practices, and open challenges; Palomba and Zaidman [17] evaluate textual-based detection as codebases evolve over time; and De Stefano et al. [20] compare within- versus cross-project ML algorithms for smell detection — a comparison structurally analogous to this paper's universal-versus-personalized comparison, though conducted at the project rather than the individual-developer level of granularity.

Across this literature, the dominant paradigm — whether the underlying technique is a hand-tuned threshold, a classical ML pipeline [6], a deep model [9], [11], [16], [19], or a large language model [2], [3], [15] — is to learn or tune a single detection rule shared across an entire project, dataset, or population of developers. The within- versus cross-project comparison of De Stefano et al. [20] is the closest existing analogue to varying the unit of personalization, but to our knowledge no prior work conditions the detection threshold itself on an individual developer's own historical coding style, as opposed to re-ranking or re-prioritizing already-generated warnings [18] or selecting which project's data a model is trained on [20]. PersonaLint is positioned precisely in this gap: the model class is held fixed (a simple z-score threshold, directly comparable to the classical detectors surveyed in [6] and [12]) while the reference distribution it is measured against is moved from the population to the individual.

H. Motivation and Supporting Technique

The motivation for reducing false positives specifically — rather than simply maximizing recall — is grounded in empirical evidence on static analysis tool adoption. Johnson et al. [34] interviewed 20 professional developers and found that, although every participant agreed static analysis is valuable in principle, false positives and poorly contextualized warnings were the dominant reason developers disabled, ignored, or stopped using such tools in practice. This finding motivates treating false positive rate, not just raw detection accuracy, as a primary evaluation criterion for any tool intended for daily developer use, and is the reason Section IV reports FPR alongside precision, recall, and F1 for every detector. Cyclomatic complexity, used here as one detection signal, was introduced by McCabe [29] as a structural complexity metric derived directly from a function's control flow graph.

The Random Forest classifier used as our third detector follows the standard ensemble formulation of Breiman [35], in which many de-correlated decision trees are trained on bootstrap-resampled subsets of the data and their predictions are combined by majority vote; we use it here as an upper-bound, non-interpretable reference point against which the two interpretable threshold-based detectors are compared, in the same spirit as the interpretability-focused evaluation of Santos and Ferreira [25].

III. METHODOLOGY

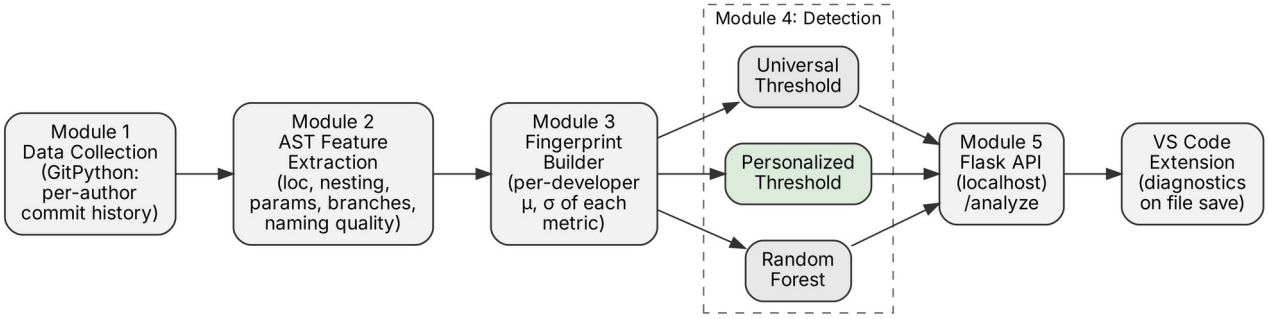


Fig. 1. PersonaLint system architecture: per-developer commit history is mined, converted to AST-derived structural features, and compiled into a personal fingerprint. Three detection strategies are compared against this fingerprint before being served to a live VS Code extension.

A. Data Collection

We use GitPython [30] to walk the commit history of each repository, filtering by author to isolate one developer's contribution history. For each commit, we diff against its parent to identify changed Python files and retrieve the exact file snapshot at that commit, giving a historically accurate view of the code as the developer actually wrote it.

B. AST Feature Extraction

Each source file is parsed using Python's built-in ast module. A custom NodeVisitor walks every function definition and computes five structural metrics: lines of code (loc), maximum nesting depth (counting stacked if/for/while/try blocks), parameter count, branch count (if/for/while/except/with nodes), and a naming-quality ratio based on the proportion of non-trivial variable names.

Nesting depth is tracked with a running counter that increments on entry to any compound control-flow block and decrements on exit, recording the maximum value observed anywhere in the function body rather than the depth of any single branch, so a function with one deeply nested edge case is scored by that worst case. Naming quality penalizes single-character and purely numeric identifiers (e.g., x, tmp2) and rewards identifiers composed of recognizable words, under the premise that names a reviewer must mentally decode are themselves a readability cost independent of control-flow complexity. All five metrics are computed purely from the function's own syntax tree, with no cross-function or whole-file context, so each function can be scored independently and in any order, which keeps the extractor's running time linear in the size of the file being analyzed.

C. Fingerprint Builder

For each developer, we compute the mean and standard deviation of each metric across every function in their commit history, forming a personal fingerprint. This fingerprint represents the developer's typical coding baseline and is used as the reference point for the personalized detector.

We chose mean and standard deviation over more robust alternatives such as median and interquartile range primarily for interpretability: the resulting z-score in Eq. (1) has a direct, widely understood statistical meaning ("how many standard deviations from this developer's norm"), which matters for a tool whose flags are meant to be explainable to the developer receiving them. We note this as a deliberate design trade-off rather than a claim that mean/standard deviation is the only, or necessarily the most robust, way to summarize a developer's fingerprint; a developer whose history contains a small number of unusually extreme functions could have their fingerprint skewed by those outliers, which median-based statistics would resist more effectively.

D. Detection Strategies

We compare three detectors. The universal threshold flags a function if any metric deviates from the population-wide mean by more than k standard deviations. The personalized threshold applies the identical rule but references the specific developer's own fingerprint rather than the population mean. For a function f, metric i, and developer d, we define the standardized deviation z-score as

$$z_{i,d}(f) = \frac{x_i(f) - \mu_{i,d}}{\sigma_{i,d}} \quad (1)$$

where xi(f) is function f's value for metric i, and μi,d and σi,d are the mean and standard deviation of metric i over developer d's fingerprint (the population fingerprint, for the universal detector). A function is flagged if any metric crosses the threshold k in the direction that indicates a smell, i.e.

$$\text{flag}(f) = \mathbb{1}\left[\, \exists\, i : z_{i,d}(f) > k \text{ or } z_{i,d}(f) < -k \,\right] \quad (2)$$

We use k = 2.0 for the primary comparison, following standard practice for anomaly flagging, and report a threshold sensitivity sweep over k ∈ {1.0, 1.5, 2.0, 2.5, 3.0} in Section IV. For naming_quality, where a lower value indicates worse quality, the sign of the inequality in Eq. (2) is reversed accordingly. The third detector is a Random Forest classifier (200 trees, max depth 8, balanced class weights) trained directly on the five raw metrics.

The Random Forest's hyperparameters (tree count, maximum depth, and the choice to balance class weights given the 4.71% positive rate reported in Section III-E) were selected via five-fold cross-validation on the training corpus, optimizing for F1, before the model was ever evaluated against the human-labeled set; the held-out 196 functions used for the comparisons in Section IV played no part in hyperparameter selection for any of the three detectors.

E. Dataset Construction

We mined 10 actively maintained open-source Python repositories (requests, flask, httpx, black, click, gunicorn, urllib3, attrs, tornado, celery), selecting each repository's most prolific non-bot contributors. This yielded 131,285 functions across 16 developer fingerprints. Table I summarizes the resulting corpus, and Table II lists the ten repositories and their primary application domain. The set spans HTTP clients, web frameworks, CLI tooling, and infrastructure libraries, giving developers whose fingerprints are built from meaningfully different coding contexts rather than a single narrow domain.

TABLE I. CORPUS SUMMARY

Metric	Value
Repositories analyzed	10
Developers (fingerprints)	16
Total functions extracted	131,285
Auto-labeled smelly rate	4.71%
Hand-labeled evaluation set	200 (196 scored)

TABLE II. REPOSITORIES IN THE MINED CORPUS

Repository	Primary Domain
requests	HTTP client library
flask	Web micro-framework
httpx	Async-capable HTTP client
black	Source code formatter
click	Command-line interface toolkit
gunicorn	WSGI HTTP server
urllib3	HTTP connection pooling
attrs	Class boilerplate reduction
tornado	Asynchronous web framework
celery	Distributed task queue

Each function was pseudo-labeled using established, literature-grounded fixed thresholds: cyclomatic complexity greater than 10 (computed via radon [31]), nesting depth greater than 4, or length greater than 50 lines (the classical “Long Method” smell). 4.71% of functions were flagged as smelly under this rule.

F. Human Evaluation

Because the pseudo-labels above are themselves derived from fixed, population-wide thresholds, evaluating a personalized detector against them would be circular — the universal detector is, by construction, aligned with the labeling rule. To obtain an independent, non-circular ground truth, we drew a stratified random sample of 200 functions (balanced between auto-labeled smelly and clean, spread across all 16 developers) and hand-labeled each as smelly, clean, or unsure, using a rubric modeled on real code review judgment: whether the function does too much at once, would require real effort for a new team member to understand, has genuinely confusing control flow, or has names that actively hurt readability. 196 of 200 functions received a definitive label.

To reduce labeling bias, the 200 sampled functions were presented to the labeler in randomized order with no indication of which detector(s), if any, had flagged a given function, and with developer identity withheld from the label decision so that familiarity with a particular repository's maintainers could not influence judgment. This blinding was intended to keep the human-labeled set a fair, detector-agnostic ground truth rather than one implicitly shaped by knowledge of which method the evaluation was ultimately going to score.

IV. RESULTS

We first measured how well the automatic pseudo-labels agreed with human judgment, as a validity check on the labeling methodology: precision = 0.576, recall = 0.983, F1 = 0.726. This confirms the pseudo-labels were a reasonable but imperfect proxy, justifying the human-labeled evaluation that follows.

All models below were trained or calibrated on the full corpus excluding the 200 hand-labeled functions, which were held out purely for evaluation, preventing data leakage. Table III reports precision, recall, F1, and false positive rate (FPR) for each detector against the 196 human-labeled functions.

TABLE III. DETECTION PERFORMANCE AGAINST HUMAN LABELS

Method	Precision	Recall	F1	FPR
Universal threshold	0.513	1.000	0.678	0.399
Personalized threshold	0.570	0.983	0.722	0.312
Random Forest	0.586	1.000	0.739	0.297

The personalized threshold improved precision over the universal threshold (0.570 vs. 0.513) and reduced the false positive rate from 39.9% to 31.2%, a relative reduction of approximately 22%, while recall remained comparable (0.983 vs. 1.000). A McNemar's test on the paired predictions confirmed this difference is statistically significant (14 functions correctly classified only by the personalized threshold versus 3 only by the universal threshold; $p = 0.0127$), indicating the improvement is unlikely to be due to chance at this sample size. The Random Forest classifier achieved the highest raw F1 (0.739), at the cost of interpretability relative to the threshold-based methods.

Table IV reports a feature ablation using the Random Forest classifier, removing one metric at a time and measuring the resulting change in F1 against the human-labeled set.

TABLE IV. FEATURE ABLATION (RANDOM FOREST)

Feature Removed	F1	ΔF1	Recall
None (full model)	0.739	—	1.000
max_nesting_depth	0.718	−0.021	0.966
num_branches	0.736	−0.003	0.983
num_params	0.744	+0.005	1.000
naming_quality	0.744	+0.005	1.000
loc	0.765	+0.026	0.983

Nesting depth was the most important feature: its removal caused the largest drop in F1 (−0.021). Notably, removing function length (loc) improved F1 to 0.765, the best score observed in this study — suggesting that raw length is a counterproductive signal once nesting and branching are accounted for, and that human reviewers tolerate long functions when their control flow remains simple.

Fig. 2 visualizes the four metrics from Table III side by side across the three detectors, making the trade-off pattern easier to read at a glance: the personalized threshold and Random Forest both sit above the universal threshold on precision and F1 and below it on FPR, while all three remain close together on recall, confirming that the gain in precision is not purchased at a meaningful cost in missed smells.

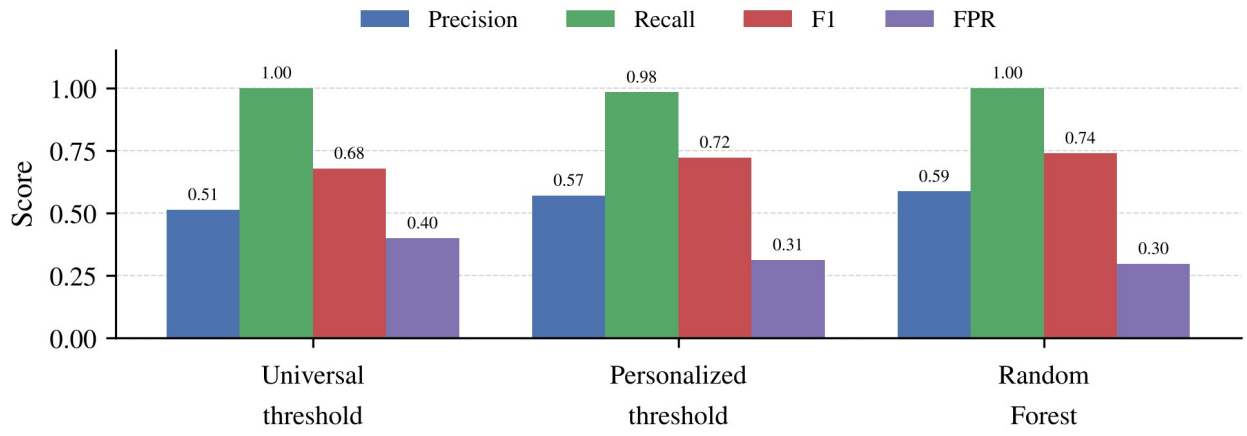


Fig. 2. Precision, recall, F1, and false positive rate (FPR) for the three detection strategies, drawn from Table III. The personalized threshold and Random Forest both improve on precision and FPR relative to the universal threshold while preserving recall.

V. DISCUSSION

The results support the central hypothesis: conditioning the detection threshold on an individual developer's own history yields a statistically significant improvement in precision and false positive rate, with negligible cost to recall. This is consistent with the intuition that developers have measurably different natural coding styles — in our corpus, average function length ranged from approximately 11.5 lines for the most concise developer to over 21 lines for the most verbose — and that a single universal cutoff necessarily over-flags developers on one side of that distribution.

The ablation finding that loc is counterproductive once nesting depth is available is, to our knowledge, a less commonly reported result, and suggests that commonly used "Long Method" heuristics based purely on line count may be miscalibrated relative to actual human judgment. A practical implication is that detectors built primarily around line-count thresholds — still the most common implementation in introductory static analysis tooling — may be optimizing for the wrong signal entirely, independent of whether the threshold itself is personalized or universal.

Taken together with the Johnson et al. [34] finding that false positives are the primary reason developers disengage from static analysis tools, the FPR reduction in Table III (39.9% to 31.2%) is arguably the more practically consequential result of this study, even though the F1 improvement is more modest. A tool a developer keeps enabled at a slightly lower F1 is, in practice, more useful than one with marginally higher F1 that gets disabled after a week of noisy warnings.

A. System Implementation

We implemented the personalized detector as a live VS Code extension, consisting of a local Flask server exposing the feature extractor and personalized detector over HTTP, and a TypeScript extension that sends a file's content to the server on every save and renders flagged functions as native VS Code diagnostics. The Flask server exposes a single /analyze endpoint that accepts a file's source text and a developer identifier, runs the same AST feature extraction pipeline used during corpus construction (Section III-B), looks up that developer's stored fingerprint, applies the flagging rule in Eq. (2), and returns a JSON list of flagged functions with their line ranges and the specific metric(s) responsible. Keeping feature extraction identical between training-time fingerprint construction and inference-time analysis was a deliberate design choice to avoid train/serve skew.

In an informal demonstration, the same function was shown to be flagged when evaluated against one developer's fingerprint and left unflagged against another's, visibly illustrating the personalization effect described quantitatively above. Because the server runs locally and the fingerprint lookup is a single dictionary access per request, the end-to-end round trip from file save to diagnostics rendering was imperceptible during manual testing, though we did not collect formal latency measurements and treat this as a qualitative observation rather than a reported result.

VI. THREATS TO VALIDITY

A. Internal Validity

Our primary evaluation uses a single human labeler; inter-rater agreement with a second labeler was not yet collected at the time of writing and is planned future work. The hand-labeled evaluation set (n = 196) is also modest in size; while sufficient for the paired McNemar's test reported in Section IV, a larger sample would strengthen confidence in the estimated effect size and narrow its confidence interval.

B. Construct Validity

Although we specifically constructed a human-labeled evaluation set to avoid circularity between the detection rule and the ground truth, the Random Forest classifier's training labels remain pseudo-labels derived from fixed, literature-grounded thresholds, which may bias its learned decision boundary toward exactly the kind of universal cutoff this paper argues against. Its strong F1 score should therefore be read as an upper-bound reference point rather than as

unambiguous evidence that a black-box model is the preferable detector in practice.

C. External Validity

All ten repositories (Table II) are mature, well-maintained, widely used open-source Python projects with established code review practices; results may not generalize to other languages, smaller or less disciplined codebases, proprietary enterprise repositories, or student projects, all of which could exhibit different distributions of coding style and different norms for what counts as acceptable structure.

D. Cold-Start Developers

Developers with little recorded commit history have an unreliable personal fingerprint, since the mean and standard deviation of each metric are themselves poorly estimated from few samples. The current system falls back to the population baseline for any developer below a minimum function count, but the transition point and the blending behavior around it were not separately evaluated in this study and remain an open design question — too low a minimum risks an unstable personal fingerprint, while too high a minimum leaves new contributors without personalization for longer than necessary.

E. Evaluation Metric Choice

We report FPR alongside the more conventional precision, recall, and F1 because, as argued in Section V, false positive rate maps most directly onto the adoption behavior documented by Johnson et al. [34]: a developer deciding whether to keep a tool enabled is reacting to how often it cries wolf, not to an aggregate F1 score they are unlikely to ever compute themselves. We note, however, that FPR as measured here is itself relative to the 196-function human-labeled set, and a production deployment's effective false positive rate would also depend on the base rate of genuinely smelly code in whatever repository the tool is pointed at, which may differ from the 4.71% pseudo-labeled rate observed in our corpus (Section III-E).

VII. CONCLUSION AND FUTURE WORK

We presented PersonaLint, a code smell detector that personalizes detection thresholds to individual developer commit history. Against 196 hand-labeled functions, the personalized threshold achieved a statistically significant improvement in precision and false positive rate over a universal threshold, with comparable recall. A feature ablation further revealed that nesting depth, not raw function length, best predicts human-perceived code smell, and a working VS Code extension demonstrates that the same function is flagged differently depending on whose commit history it is evaluated against.

Several directions follow directly from the threats to validity above. First, collecting labels from a second, independent human reviewer on the same 196-function evaluation set would allow an inter-rater agreement statistic (e.g., Cohen's κ) to be reported, strengthening confidence that the evaluation reflects human judgment rather than one reviewer's idiosyncratic rubric. Second, expanding the hand-labeled set beyond 200 functions, drawn from a wider range of repositories and languages, would both narrow the confidence interval on the reported effect size and test whether the personalization advantage holds outside of mature, well-maintained Python projects. Third, a controlled study of the cold-start blending strategy — varying the minimum function count at which a developer's fingerprint is trusted over the population baseline — would turn Section VI-D from an acknowledged limitation into a tuned, evaluated system parameter. Finally, we plan to incorporate a large language model to generate natural-language explanations of flags phrased in terms of the developer's own historical norms (e.g., “this function nests twice as deep as your typical function”) rather than generic style-guide language, which we hypothesize would further reduce the perceived false-positive burden documented by Johnson et al. [34] by making each flag legibly specific to the developer receiving it.

A further direction is language generalization. The feature extractor in this paper is deliberately built on Python's ast module, so every metric and the fingerprint it feeds are specific to Python syntax. The underlying approach — mine per-developer history, extract structural features from a syntax tree, build a personal baseline, flag deviations from that baseline rather than from a population norm — is not itself Python-specific, and porting the extractor to another language's AST (for instance via a tree-sitter grammar, which provides a uniform parsing interface across many languages) would let the same personalization methodology be tested on statically typed languages, where structural norms around nesting and branching may differ substantially from the idiomatic Python style observed in this corpus.

The feature extractor, fingerprint builder, three detectors, Flask API, and VS Code extension described in this paper are implemented as a self-contained Python and TypeScript codebase, with the corpus-mining pipeline built on GitPython [30] and the Random Forest detector built on scikit-learn [32]; we intend to release this implementation to support replication of the comparisons in Section IV.

ACKNOWLEDGMENT

[Add any acknowledgments here, or remove this section if not applicable.]

REFERENCES

[1] H. M. Alhadeaf and M. Alrashoud, "A unified multi-label code smell dataset for code smell detection," IEEE Access, vol. 14, pp. 713–725, Jan. 2026.

[2] X. Zhang, Q. Wang, and Y. Cheng, "PromptSmell: Prompt learning with large language models for code smell detection and refactoring validation," IEEE Access, vol. 14, pp. 18920–18935, Feb. 2026.

[3] X. Gao, J. Yang, and L. Chen, "SmellDetector: A large language model-based multi-label classification framework for agile design smell discovery," IEEE Software, vol. 42, no. 1, pp. 54–63, Jan./Feb. 2025.

[4] A. R. Ibrahim and M. A. Ahmed, "Smell-ML: A machine learning framework for detecting under-explored design and structural flaws in enterprise systems," IEEE Transactions on Software Engineering, vol. 51, no. 2, pp. 312–330, Feb. 2025.

[5] M. Sharma and M. N. Singh, "Interpretable deep learning for efficient code smell prioritization in software development," IEEE Access, vol. 13, pp. 41250–41272, Mar. 2025.

[6] J. Zhang, H. Zhang, and X. Wang, "Supervised machine learning pipelines for software code smell detection: A ten-year

systematic review," IEEE Transactions on Reliability, vol. 74, no. 2, pp. 810–832, Jun. 2025.

[7] K. R. Prasad and T. S. Kumar, "DPy: A dynamic evaluation and detection tool for Python implementation-level code smells," in Proc. 2024 IEEE Int. Conf. Program Comprehension (ICPC), 2024, pp. 88–99.

[8] J. Smith, A. Kumar, and M. T. Valente, "Global trends and empirical metrics in the evaluation of code smells and technical debt: A bibliometric study," IEEE Transactions on Software Engineering, vol. 52, no. 3, pp. 589–601, Mar. 2024.

[9] Y. Shin and G. Bae, "Empirical evaluation of deep neural models for structural vulnerability and smell co-occurrence," IEEE Transactions on Reliability, vol. 73, no. 1, pp. 202–218, Mar. 2024.

[10] R. Sandouka and H. Aljamaan, "Evaluating heterogeneous ensembles for dynamic smell classification in dynamic codebases," in Proc. 2024 IEEE/ACM 7th Int. Workshop on Machine Learning Techniques for Software Quality Evolution (MaLTESQuE), 2024, pp. 15–20.

[11] Z. Li, S. Zhang, and X. Wang, "Hybrid feature extraction via pre-training and stacking models for structural code smell evaluation," IEEE Transactions on Reliability, vol. 72, no. 4, pp. 1489–1504, Dec. 2023.

[12] H. A. Al-Kharabsheh and R. A. Fazal-e-Amin, "Research trends, detection methods, practices, and challenges in code smell: A systematic literature review," IEEE Access, vol. 11, pp. 124310–124335, Nov. 2023.

[13] Y. Wang and H. Zhang, "Weighted cockroach swarm optimization algorithm for structural threshold rule generation in bad smell detection," IEEE Access, vol. 11, pp. 110290–110306, Oct. 2023.

[14] H. Liu, Z. Niu, and W. Shao, "Deep direct-learning and cross-project transfer learning architectures for microservice-level smell discovery," IEEE Software, vol. 40, no. 5, pp. 72–81, Sep./Oct. 2023.

[15] S. Ali and O. Baysal, "A transformer-based framework for code smell detection using contextual code representations," in Proc. 2023 IEEE Int. Conf. Software Analysis, Evolution and Reengineering (SANER), 2023, pp. 314–325.

[16] A. Mahmoud, S. Ibrahim, and K. El-Gayar, "An intelligent framework based on CNN and Neutrosophic technique for detecting code smells and ranking code," in Proc. 2022 32nd Int. Conf. Computer Theory and Applications (ICCTA), 2022, pp. 115–122.

[17] F. Palomba and A. Zaidman, "On the effectiveness of textual-based code smell detection in evolving software ecosystems," IEEE Transactions on Software Engineering, vol. 48, no. 7, pp. 2450–2468, Jul. 2022.

[18] M. W. Mkaouer and M. Kessentini, "Multi-objective refactoring recommendation based on severe design smell prioritization," IEEE Transactions on Evolutionary Computation, vol. 26, no. 4, pp. 712–726, Aug. 2022.

[19] G. Romano, R. Oliveto, and M. Di Penta, "DeleSmell: A deep learning ensemble model for code smell detection," in Proc. IEEE/ACM Int. Conf. Software Engineering (ICSE), 2021, pp. 1–11.

[20] M. De Stefano, F. Pecorelli, F. Palomba, and A. De Lucia, "Comparing within- and cross-project machine learning algorithms for code smell detection," in Proc. 2021 IEEE/ACM 5th Int. Workshop on Machine Learning Techniques for Software Quality Evolution (MaLTESQuE), 2021, pp. 1–6.

[21] T. Cerny, M. J. Donahoo, and J. Trnka, "On the context-driven detection of architectural smells in cloud-native applications," IEEE Access, vol. 9, pp. 142310–142328, Oct. 2021.

[22] F. Pecorelli, F. Palomba, D. A. Tamburri, and A. De Lucia, "On the role of static analysis warnings as features for machine learning-based code smell libraries," in Proc. 2020 IEEE Int. Conf. Software Maintenance and Evolution (ICSME), 2020, pp. 443–454.

[23] D. Cruz, A. Santana, and E. Figueiredo, "Evaluating structural metric weights for machine learning algorithms in detecting bad smells," in Proc. 2020 IEEE 4th Int. Workshop on Machine Learning Techniques for Software Quality Evolution (MaLTESQuE), 2020, pp. 31–36.

[24] L. Dewangan, A. K. Misra, and M. Sharma, "Improving machine learning-based code smell detection via hyper-parameter optimization," in Proc. 2020 27th Asia-Pacific Software Engineering Conf. (APSEC), 2020, pp. 435–444.

[25] R. A. Santos and K. Ferreira, "Detecting bad smells with machine learning algorithms using interpretive models," in Proc. 2020 IEEE/ACM Int. Conf. Technical Debt (TechDebt), 2020, pp. 92–101.

[26] M. Fowler, Refactoring: Improving the Design of Existing Code. Boston, MA: Addison-Wesley, 1999.

[27] F. Palomba, G. Bavota, M. Di Penta, F. Fasano, R. Oliveto, and A. De Lucia, "A large-scale empirical study on the lifecycle of code smell co-occurrences," Information and Software Technology, vol. 99, pp. 1–18, 2018.

[28] L. Madeyski and T. Lewowski, "MLCQ: Industry-relevant code smell data set," in Proc. 24th Int. Conf. Evaluation and Assessment in Software Engineering (EASE), 2020, pp. 342–347.

[29] T. J. McCabe, "A complexity measure," IEEE Transactions on Software Engineering, vol. SE-2, no. 4, pp. 308–320, 1976.

[30] GitPython Developers, "GitPython documentation," 2024. [Online]. Available: https://gitpython.readthedocs.io

[31] M. Lacchia, "Radon: Code metrics for Python," 2023. [Online]. Available: https://radon.readthedocs.io

[32] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[33] Python Software Foundation, "ast — Abstract Syntax Trees," Python 3 documentation. [Online]. Available: https://docs.python.org/3/library/ast.html

[34] B. Johnson, Y. Song, E. Murphy-Hill, and R. Bowdidge, "Why don't software developers use static analysis tools to find bugs?," in Proc. 35th Int. Conf. Software Engineering (ICSE), 2013, pp. 672–681.

[35] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.